DATABRICKS × FULTON HOGAN · ENTERPRISE DATA PLATFORM

SEMANTIC MODEL REFRESH

Production-ready, field-validated architecture for SLA-driven data freshness across 7+ semantic models in the Enterprise Data Platform.

FEBRUARY 2026 · v2.0 VALIDATED DESIGN

01 / PROBLEM STATEMENT

CURRENT STATE & BUSINESS IMPACT

EDP supports 7+ semantic models across AWM, Finance, Operations and more — each with different freshness expectations and business criticality.

01

Inconsistent Freshness

- Same Gold tables appear across models with no standardised refresh behaviour
- Business users cannot determine which version of a report reflects current data

02

Mismatched Cadence vs Need

- Low-change tables refreshed too frequently — wasting compute and increasing cost
- Near-real-time tables such as Dispatch may lag, directly impacting operational decisions

03

Tight Coupling, No Isolation

- Refresh logic must live at the platform level, not inside individual semantic models
- Without isolation, a failure in one model cascades and blocks all others in a shared job

OUTCOME

Operational noise from failed and late refreshes · Trust risk in reporting — which dashboard reflects current data? · Rising platform costs from over-refreshing low-change data and failures without isolation.

02 / ROOT CAUSE

PLATFORM-LEVEL GAPS & RISKS

01 No Centralised Refresh Design

No single place classifies tables into NRT / Medium / Daily / On-Demand or enforces SLA expectations consistently. Without a centralised framework, every model manages its own rules independently and drift is inevitable.

02 Monitoring and Governance Gaps

No visibility into last refresh timestamp, duration, failure reasons or SLA compliance per model. There is no audit trail, and teams investigate manually to determine whether data loads succeeded or which model is responsible for a delay.

03 Scalability and Extensibility Risk

As new models are added — currently 7+ and growing — refresh rules are not automatically inherited from the platform. Each onboarding requires redesigning logic in multiple places, with increasing cost and risk of inconsistency over time.

03 / SOLUTION DESIGN

DESIGN PRINCIPLES

Six validated principles that drive every architectural decision in this framework.

1

Model-level refresh

One Power BI refresh per model — never per table. Aligns with how Power BI actually works: datasets refresh as an atomic unit. Table-level granularity requires Premium XMLA endpoints and adds unnecessary complexity.

2

One Workflow per model per cadence

Separate Databricks Workflows per model and cadence: AWM-DAILY, Finance-DAILY, AWM-TABLE_TRIGGERED. Failure in one model never blocks another. Retry logic, timeouts and alerting are independently tuned per model.

3

Native table update triggers for NRT

Databricks native table update trigger fires when a Gold table changes — no polling, no wasted compute. Built-in debounce via `wait_after_last_change_seconds` replaces custom interval logic entirely.

4

SLAs as data, not code

`sla_threshold_minutes` lives in the Unity Catalog metadata table. Changing a threshold is a SQL UPDATE — no code deployment, no pipeline change. Delta history provides a full audit trail of every SLA adjustment.

5

Power BI task — primary integration

The native Databricks Power BI task type handles standard refreshes without custom auth code. The REST API is available as a fallback for edge cases only: cross-workspace batch operations and complex conditional logic.

6

Cost and freshness tracking built in

Every refresh execution writes duration, status, warehouse DBU consumed and SLA breach flags to Unity Catalog Delta tables from day one. Observability is a first-class design requirement — not an afterthought.

Guiding principle: Databricks owns table-level data freshness and orchestration. Semantic models own shaping, measures and serving.

04 / TECHNICAL — METADATA

UNITY CATALOG METADATA SCHEMA

All refresh behaviour is configuration, not code. Four Delta tables in `edp_metadata.semantic_refresh` drive the entire framework.

semantic_models

Registry of all Power BI semantic models

- **model_id, model_name**
  Unique identifier and display name for the model
- **workspace_id, dataset_id**
  Power BI GUIDs required for task and API calls
- **connection_name**
  UC Power BI connection, e.g. edp_powerbi.awm_workspace
- **query_mode**
  Import / DirectQuery / Dual / Hybrid — drives refresh behaviour
- **sla_tier**
  T0 = critical, T1 = important, T2 = standard — controls alert priority
- **auth_mode, owner_email**
  SERVICE_PRINCIPAL / OAUTH / PAT — and operational contact

table_refresh_config

Per-table cadence and trigger rules

- **config_id, model_id**
  Primary key and FK to semantic_models
- **source_table**
  Fully qualified UC table: catalog.schema.table
- **refresh_cadence**
  NEAR_REALTIME / MEDIUM_FREQUENCY / DAILY / ON_DEMAND
- **trigger_mode**
  SCHEDULE / TABLE_UPDATE / PIPELINE_COMPLETE / MANUAL
- **sla_threshold_minutes**

- Max minutes before breach — data-driven, SQL-updateable
- **`refresh_type, is_active`**
  FULL / INCREMENTAL — and enable/disable flag per table

`refresh_execution_log`

Audit log of every refresh run

- **`execution_id, model_id`**
  Unique run identifier and FK to semantic_models
- **`workflow_run_id`**
  Databricks job run ID for end-to-end tracing
- **`powerbi_refresh_id`**
  Power BI API response ID for cross-service correlation
- **`status`**
  RUNNING / SUCCESS / FAILED / TIMEOUT
- **`duration_seconds, rows_processed`**
  Performance metrics per run for health scoring
- **`triggered_by, error_message`**
  Trigger source and error detail for incident triage

`refresh_cost_tracking`

DBU and USD cost per refresh run

- **`cost_id, execution_id`**
  FK to refresh_execution_log
- **`warehouse_id, warehouse_name`**
  SQL warehouse that served the refresh request
- **`compute_units_consumed`**
  DBU consumed per run — source for FinOps reporting
- **`warehouse_cost_estimate_usd`**
  Estimated cost in USD per refresh execution
- **`query_count, data_transfer_gb`**
  Query-level metrics for transfer volume to Power BI
- **`compute / storage / network split`**
  Cost breakdown for detailed FinOps attribution

All four tables have **Delta Change Data Feed enabled** — full audit history of every config change, refresh run and cost entry. The execution log is **partitioned by date** for efficient time-range querying. Log retention: 90 days.

05 / TECHNICAL — ORCHESTRATION

WORKFLOW ARCHITECTURE

One Databricks Workflow per model per cadence — complete failure isolation and independent operational control per model.

OPTION A — CONSIDERED Not selected
Cadence-based shared jobs

```
Workflow: "Daily Refresh"
  ├── AWM     (3 tables)
  ├── Finance  (5 tables)
  └── Operations (4 tables)
```

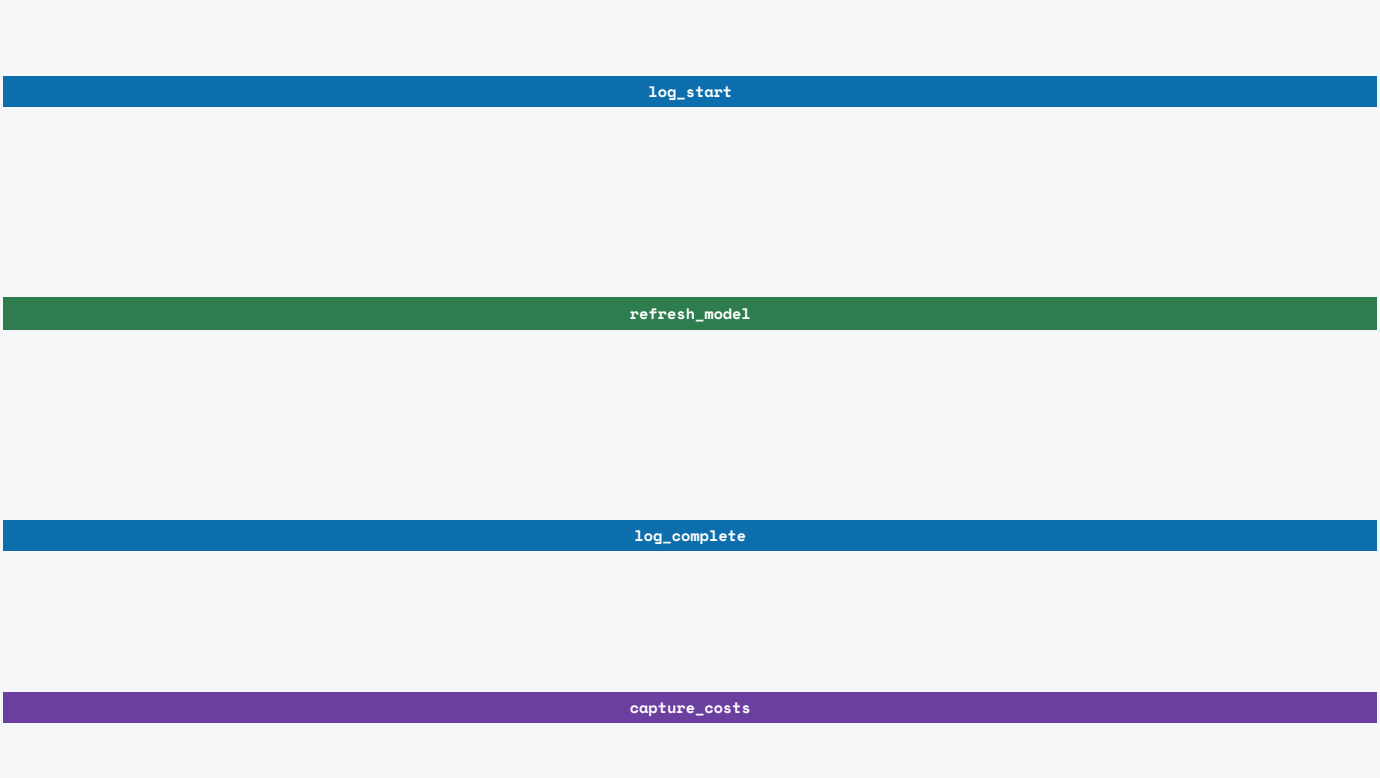
− A Finance failure blocks AWM and Operations. SLA ownership is unclear. Retry and timeout settings are shared. Difficult to isolate which model caused an incident or to tune behaviour per model.

OPTION B — SELECTED CHOSEN
Model-based isolated jobs

```
AWM-DAILY            (cron 06:00 NZT)
AWM-MEDIUM_FREQUENCY  (every 4h)
AWM-TABLE_TRIGGERED   (on gold.awm.fact_dispatch)
Finance-DAILY         (cron 06:00 NZT)
Finance-MEDIUM_FREQUENCY(every 4h)
```

+ Complete failure isolation — a Finance issue never blocks AWM. Each model has independent retry count, timeout and alerting configuration. Onboarding a new model generates one new workflow with no changes to existing jobs.

TASK SEQUENCE — EACH MODEL WORKFLOW



WORKFLOW GENERATION — PYTHON SDK

```
# 05_generate_workflows_model_based.py scheduled = spark.sql("""
SELECT m.model_id, m.model_name, m.connection_name,
m.dataset_id, m.sla_tier, c.refresh_cadence, COLLECT_LIST(c.table_name) AS tables FROM semantic_models m JOIN
table_refresh_config c ON m.model_id = c.model_id WHERE m.is_active = true AND c.trigger_mode = 'SCHEDULE' GROUP BY
m.model_id, c.refresh_cadence """).collect() for model in scheduled: name = f"{model.model_name} - {model.refresh_cadence}"
w.jobs.create( name = name, tasks = [{ "task_key": "refresh_model", "power_bi_task": { "connection_name":
model.connection_name, "dataset_id": model.dataset_id, "refresh_type": "full" } }], tags = {"ManagedBy": "MetadataFramework"}
)
```

06 / TECHNICAL — TRIGGERS

TRIGGERS & REFRESH MECHANICS

How near-real-time refresh is triggered, how the Power BI task integrates, and when the REST API fallback applies.

OPTION A — CONSIDERED Not selected
Scheduled polling every 15 minutes

```
# Runs on fixed schedule regardless of data changes
for table in near_realtime_tables:
  last = spark.sql(
    f"DESCRIBE HISTORY {table}").first()
  if last > last_refresh:
    trigger_refresh()
```

— Compute runs every 15 minutes whether or not data has changed. Polling interval creates artificial latency. Requires maintaining custom notebook logic and table history tracking.

OPTION B — SELECTED CHOSEN
Native Databricks table update trigger

```
"trigger": {
  "table_update": {
    "table_names": ["gold.awm.fact_dispatch"],
    "condition": "ANY_UPDATE",
    "wait_after_last_change_seconds": 1800
  }
}
```

No wasted compute
Workflow runs only when a Gold table change is detected — never on a fixed timer

Lower latency
Trigger fires immediately on change — no waiting for the next poll window to expire

Built-in debounce
wait_after_last_change_seconds enforces the minimum refresh interval natively

Zero custom code
Native Databricks feature — no polling notebook or history-tracking logic to maintain

---

POWER BI TASK — PRIMARY (90% OF CASES)

Native Databricks task type — no custom authentication code required. Configured directly in the Workflow task definition alongside retry, timeout and alert settings.

- Accepts: `connection_name`, `dataset_id`, `refresh_type: full`
- Handles Import, Incremental, Scheduled and Table-triggered workflows
- Retry count, timeout and on_failure email configured directly on the task

REST API FALLBACK — EDGE CASES ONLY (10%)

```
POST https://api.powerbi.com/v1.0/myorg/groups/
      {workspaceId}/datasets/{datasetId}/refreshes
Authorization: Bearer {entra_oauth2_token}
```

- Auth: Service Principal → Entra ID OAuth2 → Bearer token via Key Vault secret scope
- Use for: cross-workspace batch operations, complex conditional logic, SSO DirectQuery edge cases
- Not required for standard Import or Incremental refresh — the Power BI task handles those

07 / TECHNICAL — OBSERVABILITY

HEALTH SCORECARD

Built in from Day 1. Every refresh execution writes to UC Delta tables — dashboards and alerts derive from this single source of truth.

Health Score — 0 to 100 per model

50% weight	Success rate over last 7 days
50% weight	SLA breach count — 0 breaches = full 50 pts, >5 breaches = 0 pts

Status Thresholds

Healthy	0 SLA breaches AND success rate ≥95%
Warning	1–5 breaches OR success rate <95%
Critical	>5 breaches OR model has never completed a refresh

Last refresh timestamp
Per model and per table, available in real time

Refresh duration
Average, maximum and minimum in seconds

SLA breach count
7-day rolling window, flagged per model

Cost per refresh
DBU consumed and projected annual USD

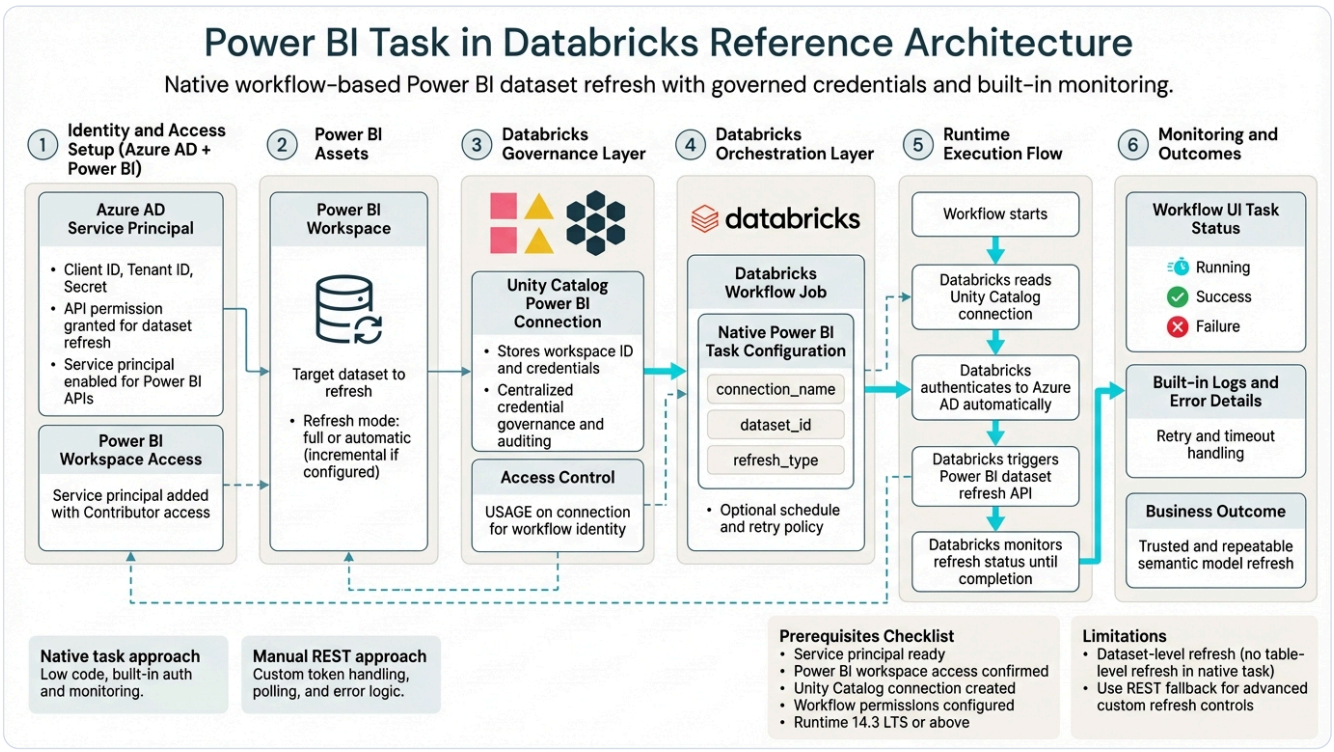
MODEL HEALTH REPORT — SQL ANALYTICS DASHBOARD QUERY

```
-- model_health_report.sql SELECT model_name, sla_tier, query_mode, CAST( (success_rate_pct * 0.5) + (CASE WHEN
sla_breaches_7d = 0 THEN 50 WHEN sla_breaches_7d <= 2 THEN 30 ELSE 0 END) AS INT) AS health_score, total_refreshes_7d,
success_rate_pct, sla_breaches_7d, avg_duration_min, total_cost_7d_usd, ROUND(total_cost_7d_usd * 52.0 / 7, 2) AS
projected_annual_cost_usd, last_refresh_timestamp, minutes_since_last_refresh, status -- Healthy / Warning / Critical FROM
model_metrics ORDER BY CASE sla_tier WHEN 'T0' THEN 1 WHEN 'T1' THEN 2 ELSE 3 END, health_score ASC;
```

07A / REFERENCE DIAGRAM

POWER BI TASK IN DATABRICKS REFERENCE ARCHITECTURE

Native workflow-driven dataset refresh using Unity Catalog connection governance and built-in execution monitoring.



Click the diagram to open the published immersive page: pravinva.github.io/bricksmith/FH_Semantic_Refresh_immersive.html

08 / DEPLOYMENT

ROLLOUT PLAN & DEPLOYMENT CHECKLIST

PHASE 1

PILOT

Weeks 1–2

- Stand up metadata schema in edp_metadata.semantic_refresh
- Seed one daily and one four-hourly model (AWM) with full config
- Generate model-based workflows via 05_generate_workflows.py
- Validate Power BI task refresh end-to-end in non-production
- Confirm execution log and cost tracking tables are populating correctly

PHASE 2

SCALE

Weeks 3–6

- Classify and seed all tables across 7+ models with cadence and SLA config
- Generate all scheduled workflows per model via the metadata-driven generator
- Build and publish the SQL Analytics health dashboard
- Configure SLA breach alerting via Teams webhook and email

PHASE 3

NRT

Post go-live

- Enable native table update triggers for validated NRT domains
- AWM Dispatch: gold.awm.fact_dispatch with 30-minute debounce
- Monitor SLA compliance and tune thresholds via metadata SQL updates
- Document operational procedures and train the operations team

Pre-Deployment Checklist

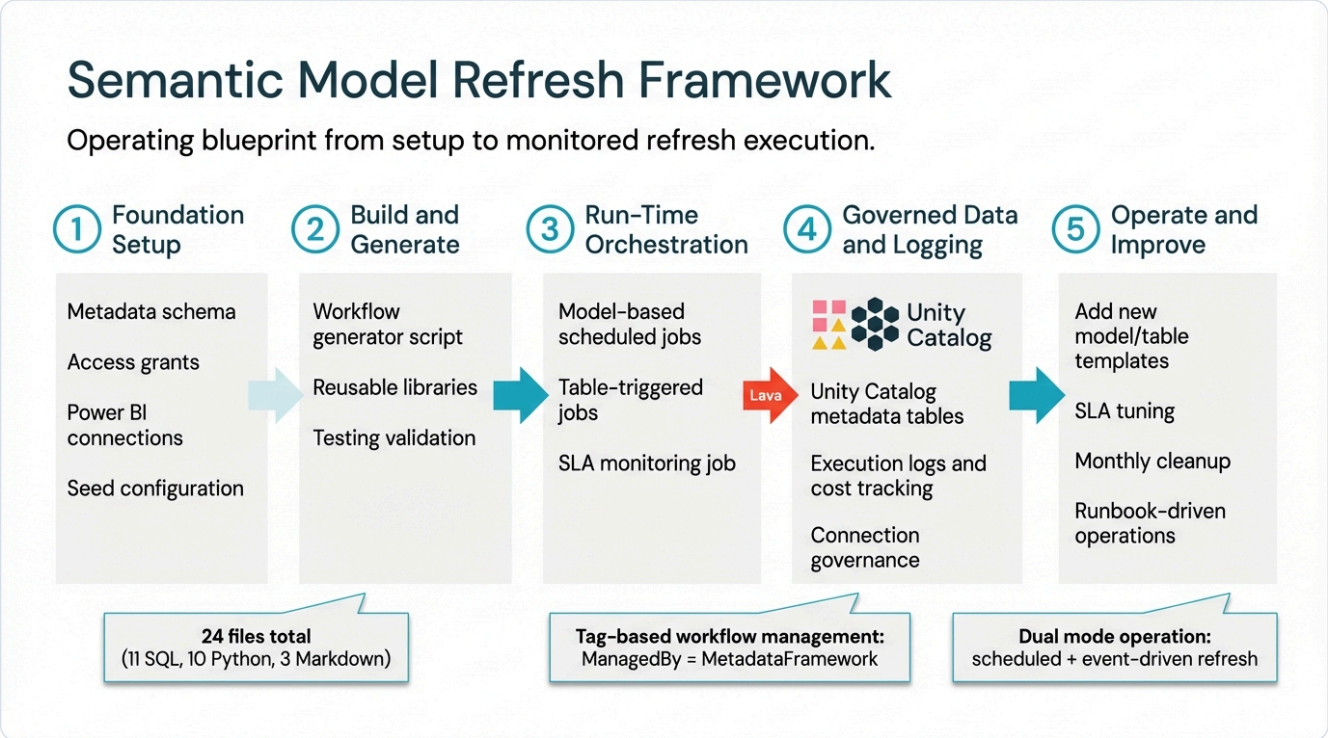
- [ ] Azure AD Service Principal created and added to Power BI workspaces with Contributor role

- [ ] Power BI Admin portal: Service principals allowed for API access
- [ ] Power BI Admin: Enhance admin APIs enabled for workspace-level operations
- [ ] Unity Catalog Power BI connections created and connection test verified for each workspace
- [ ] Metadata schema deployed and permissions granted to Service Principal and field engineers

09 / IMPLEMENTATION

SEMANTIC MODEL REFRESH IMPLEMENTATION BLUEPRINT

Practical operating blueprint from setup through workflow runtime, governance, and ongoing operations.



Implementation view covering setup, generation, orchestrated refresh execution, governed logging, and operational lifecycle.

Open detailed directory structure page | Open quick reference page

DATABRICKS × FULTON HOGAN

Enterprise Data Platform · Semantic Model Refresh v2.0 · February 2026